Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №6
на тему:
«MONGODB ОСНОВЫ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1. Написать скрипт (на любом языке), который читает CSV файлы, создает коллекции users, categories, products, orders
2. Написать запросы, проверяющие целостность данных:
    a. Подсчитать количество документов в каждой коллекции
    b. Убедиться, что все заказы имеют валидные user_id (проверка через $lookup)
    c. Найти заказы с общей суммой > 1000 (проверка расчетов)
3. Реализовать функции, аналогичные SQL:
    a. get_user_orders(user_id) - аналог SELECT * FROM orders WHERE user_id = ?
    b. get_order_with_details(order_id) - аналог JOIN orders + order_items + products
    c. get_top_products_by_revenue(limit) - аналог GROUP BY product_id ORDER BY SUM(quantity*price) DESC
4. Провести проверку производительности
    a. Создать индексы: db.orders.createIndex({ user_id: 1, created_at: -1 })
    b. Сравнить время выполнения запроса до и после создания индекса (использовать explain("executionStats"))

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

MongoDB является документоориентированной системой управления базами данных класса NoSQL. В отличие от реляционных СУБД, где данные хранятся в таблицах с жестко заданной схемой, MongoDB хранит данные в формате, близком к JSON (BSON). Основными понятиями здесь являются база данных, коллекция (аналог таблицы) и документ (аналог строки). Внутри документов существуют поля, а сами документы объединяются в коллекции, что и позволяет так или иначе структурировать неструктурированные данные. Структура MongoDB показана на рисунке 2.1.

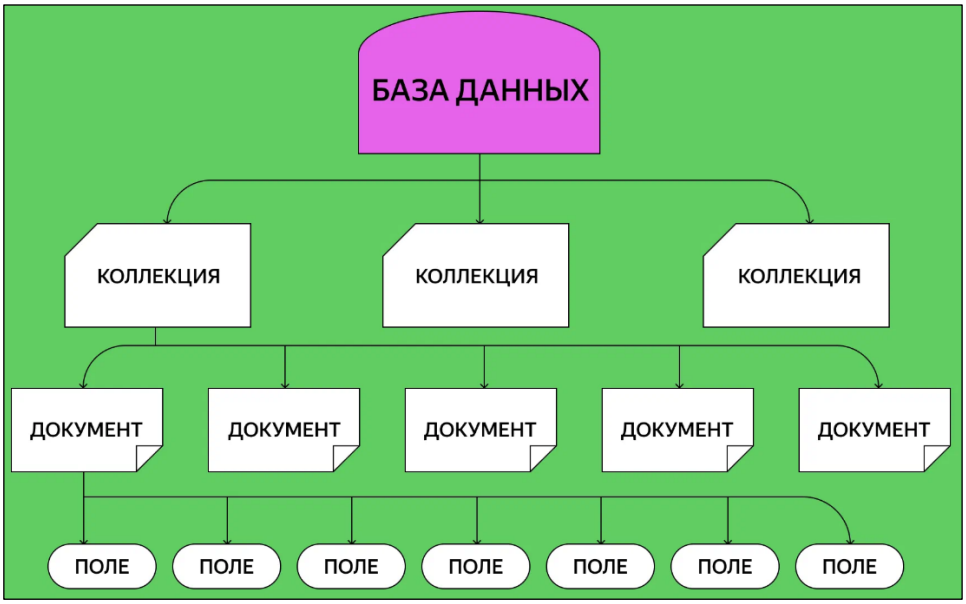

Рисунок 2.1 – Структура MongoDB

Важным преимуществом MongoDB является возможность выполнения агрегационных запросов с помощью конвейера (aggregate), который позволяет последовательно преобразовывать документы, используя такие стадии, как $lookup (аналог SQL JOIN), $group, $sort и $match. Для оптимизации производительности запросов в MongoDB используются индексы, которые позволяют значительно сократить количество просматриваемых документов и избежать дорогостоящих операций сортировки или полного сканирования коллекции.

3 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы был разработан скрипт миграции данных из CSV-файлов в базу данных MongoDB. Скрипт читает файлы с данными о пользователях, категориях, товарах, заказах и позициях заказов, приводит типы данных к необходимым (целые числа, числа с плавающей точкой, даты) и загружает документы в соответствующие коллекции users, categories, products, orders и order_items. Рисунок 3.1 показывает результат миграции данных из CSV-файлов.

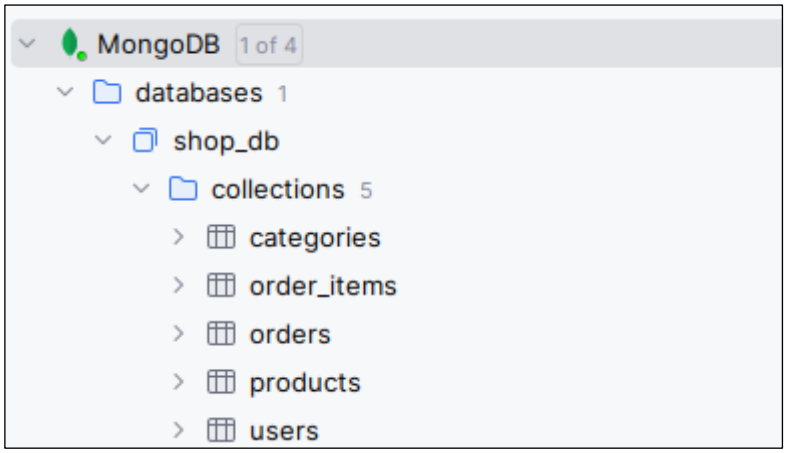

Рисунок 3.1 – Заполненная MongoDB

После выполнения миграции была запущена программа, реализующая меню для выполнения различных аналитических запросов. Первым делом была проведена проверка целостности данных (пункт 2 задания), результаты которой отображены на рисунке 3.2. Скрипт подсчитал количество документов в каждой коллекции: 10 пользователей, 8 категорий, 20 товаров, 15 заказов и 29 позиций в заказах. Также с помощью оператора $lookup была выполнена проверка внешних ключей, которая показала, что заказов с некорректными (отсутствующими) user_id не обнаружено, что показывает ссылочную целостность на уровне приложения.

```
Проверка целостности:
 - Документов в users: 10
 - Документов в categories: 8
 - Документов в products: 20
 - Документов в orders: 15
 - Документов в order_items: 29
 - Заказов с некорректными user_id: 0
```

Рисунок 3.2 – Проверка целостности базы данных

Далее был выполнен запрос на поиск заказов, общая стоимость которых превышает 1000 условных единиц. Для этого использовалась агрегация по коллекции order_items с группировкой по order_id и вычислением суммы умножения количества на цену. В результате были найдены четыре заказа с номерами 7, 2, 14 и 11, суммы которых варьируются от 1199.99 до 1699.98 (рисунок 3.3).

```
Заказы на сумму > 1000:
 - ID Заказа: 7, Сумма: 1369.97
 - ID Заказа: 2, Сумма: 1259.98
 - ID Заказа: 14, Сумма: 1199.99
 - ID Заказа: 11, Сумма: 1699.98
```

Рисунок 3.3 – Поиск заказов с общей суммой больше 1000

Следующим шагом была реализация функции, аналогичной SQL-запросу с группировкой и сортировкой – получение топа товаров по выручке. При вводе лимита, равного 3, программа выполнила конвейер, который сгруппировал данные по product_id, вычислил выручку, отсортировал результаты по убыванию и подтянул названия товаров из коллекции products. Лидерами продаж стали MacBook Air M2 с выручкой 2399.98, iPhone 15 Pro (1799.98) и Samsung Galaxy S24 (1599.98). Результат работы функции на рисунке 3.4.

```
Выберите действие: 3
Введите лимит (например, 5): 3
Товар: MacBook Air M2 | Выручка: 2399.98
Товар: iPhone 15 Pro | Выручка: 1799.98
Товар: Samsung Galaxy S24 | Выручка: 1599.98
```

Рисунок 3.4 – Получение топа товаров по выручке

Для проверки производительности индексов был проведен специальный тест. В коллекцию orders было сгенерировано 100000 тестовых документов. Затем был выполнен поиск заказов конкретного пользователя с сортировкой по дате. Без индекса запрос выполнялся 53 миллисекунды, при этом было изучено 100015 документов, и в плане выполнения фигурировала стадия сортировки (SORT). После создания составного индекса по полям user_id (по возрастанию) и created_at (по убыванию) время выполнения запроса сократилось до 43 миллисекунд и количество изученных документов сократилось до 10023, но тип сканирования изменился на FETCH, что означает эффективное использование индекса для поиска и сортировки. Результат работы показан на рисунке 3.5.

```
Тест производительности индекса:
Генерация 100000 тестовых документов...

--- БЕЗ ИНДЕКСА ---
   Время выполнения: 53 ms
   Изучено документов: 100015
   Тип сканирования: SORT

Создание индекса...

--- С ИНДЕКСОМ ---
   Время выполнения: 43 ms
   Изучено документов: 10023
   Тип сканирования: FETCH
```

Рисунок 3.5 – Тестирование производительности индексов

В ходе работы также были протестированы функции, имитирующие простые SQL-запросы. Рассмотрим функцию посложнее, а именно, функция получения деталей заказа. Для заказа №1 успешно выполнила присоединение трех коллекций, отобразив не только статус заказа, но и список купленных товаров с их количеством и ценой. Всё как на рисунке 3.6.

```
Введете ID заказа: 1


Детали заказа №1 (Статус: completed):
 - iPhone 15 Pro: 1 шт. x 899.99
 - Футболка хлопковая: 2 шт. x 19.99
```

Рисунок 3.6 – Получение деталей заказа

ВЫВОД

В результате выполнения лабораторной работы были решены задачи миграции данных из CSV-файлы в документоориентированную структуру MongoDB. MongoDB с помощью агрегационного конвейера позволяет реализовывать запросы любой сложности, включая проверку целостности данных, фильтрацию, сортировку, группировку и соединение коллекций. Эксперимент с производительностью наглядно продемонстрировал важность индексов: применение составного индекса позволило оптимизировать выполнения запроса, уйти от стадии сортировки в памяти и незначительно, но стабильно сократить время выполнения на большом объеме данных.